Scientific Machine Learning

& Simulation Intelligence

Lecture Notes

Enrico Savorgnan & Some LLM
Data Science and Artificial Intelligence
A.Y. 2026/2027

October 13, 2026

Contents

1 Differential Equations and the Finite Element Method 1

1.1 Introduction 1

1.2 Ordinary Differential Equations 1

1.3 Partial Differential Equations 3

1.4 Strong vs. Weak Formulations 4

1.5 Finite Element Method 5

1.6 Problem Formulation 6

1.7 General FEM formulation 7

2 Linear Reduced-Order Modelling 9

2.1 Introduction 9

2.2 Reduced-Basis Method 10

2.3 Proper Orthogonal Decomposition 13

2.4 Greedy Sampling 15

2.5 An Example 16

3 Nonlinear Reduced-Order Models 19

3.1 Introduction 19

3.2 Nonlinear Problems 19

3.3 Discrete Empirical Interpolation Method 20

3.4 Reduced Models for Navier–Stokes 21

3.5 Non-Intrusive ROMs 22

3.6 Error and Validation 23

3.7 Summary 24

Differential Equations and the Finite Element Method

1

1.1 Introduction

Table of Contents

1. Introduction
2. Ordinary Differential Equations
3. Partial Differential Equations
4. Strong vs. Weak Formulations
5. Finite Element Method
6. Problem Formulation
7. General FEM Formulation

Simulation science uses mathematical models to predict how physical systems evolve, from fluid flow around an aircraft to ocean circulation. These predictions support engineering design, weather forecasting, and environmental assessment. These laws express general balances: changes in mass, momentum, and energy within a region are accounted for by fluxes across its boundary and by sources. Applied to an infinitesimal control volume, the balances lead to partial differential equations (PDEs); in fluid dynamics, the coupled balances give rise to the Navier–Stokes equations. The control-volume sketch illustrates this local balance.

High-fidelity PDE simulations can be costly, especially when they must be repeated for many parameters, geometries, or operating conditions. Scientific machine learning (SciML) combines physical models with learning methods to approximate these repeated calculations. Surrogate models, including reduced-order models (ROMs) and neural operators, learn parameter-to-solution mappings such as $\boldsymbol{\mu} \mapsto u(t_k, x_i; \boldsymbol{\mu})$ and aim to evaluate them much faster. Adjoint methods also help calibrate model coefficients, for example by matching turbulence models to reference simulations.

These notes build from differential equations, numerical discretization, and differential programming (including automatic differentiation), then introduce physics-informed surrogates such as Neural ODEs, ROMs, physics-informed neural networks (PINNs), and neural operators. They also cover simulation-based inference for unknown parameters, uncertainty estimation, and causal reasoning to distinguish physical mechanisms from spurious correlations. Recommended texts and online resources supplement the lectures; assessment is through a scientific project on a course topic.

1.2 Ordinary Differential Equations

Equations involving derivatives represent how physical quantities evolve in space and time. They are universally employed to model dynamical systems across science, engineering, finance, and beyond.
Differential equations are broadly classified into two categories:

A. **Ordinary Differential Equations (ODEs)**: involving derivatives with respect to a single independent variable;

B. **Partial Differential Equations (PDEs)**: involving partial derivatives with respect to multiple independent variables (such as space

and time).

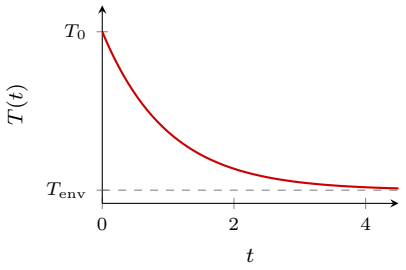


Figure 1.1: Exponential temperature decay towards T_{env} governed by Newton's Law of Cooling.

An Ordinary Differential Equation involves derivatives with respect to only one independent variable, typically time t:

$$\frac{dy}{dt} = f(y, t). \tag{1.1}$$

A classical example is **Newton's Law of Cooling**: $\frac{dT}{dt} = -k(T - T_{\text{env}})$, where $T(t)$ represents the temperature of the body, T_{env} denotes the temperature of the surrounding environment, and $k > 0$ is a positive heat transfer coefficient.

Crucial elements in the analysis of ODEs include the initial conditions, the vector field (which assigns velocity vectors in the state space), and some other conditions, e.g., on small intervals of space or time.

1.2.1 Analytical Methods for ODEs

Analytical methods provide exact, closed-form expressions for the solution of differential equations.
Common techniques include:

1. **Separation of Variables**: applicable when variables can be isolated algebraically on opposite sides of the equation.
2. **Integrating Factor**: standard for linear first-order ODEs of the form $\frac{dy}{dt} + p(t)y = q(t)$.
3. **Characteristic Equation**: employed for linear ODEs with constant coefficients.
4. **Fourier and Laplace Transforms**: powerful for translating differential equations into algebraic relationships in frequency/complex domains.

Analytical methods only work for restricted classes of equations; they quickly become intractable for nonlinear systems or complex couplings.

Example: Solve the linear growth equation: $\frac{dy(x)}{dx} = ky(x)$.
Separating variables yields $\frac{1}{y}dy = kdx$, and integrating both sides produces $y(x) = Ce^{kx}$, where C is an integration constant determined by initial conditions. This model underpins radioactive decay ($k < 0$), unconstrained population growth ($k > 0$), and simple financial compound interest models.

1.2.2 Numerical Methods for ODEs

When closed-form analytical solutions are unavailable, numerical methods approximate the continuous solution by calculating values at discrete time instances $t_n = t_0 + n\Delta t$.
Standard approaches include:

- **Euler's Method**: the simplest explicit step-by-step approximation:

$$y_{n+1} = y_n + \Delta t\, f(y_n, t_n). \tag{1.2}$$

- **Runge-Kutta Methods**: multi-stage schemes (such as the classical fourth-order RK4) that achieve high-order accuracy by sampling intermediate slopes.
- **Finite Difference Methods**: replacing derivatives with discrete quotient approximations, widely used also for boundary value problems and PDEs.

These methods are usually applicable to complex, coupled, and highly nonlinear systems, but solutions are approximations subject to truncation and round-off errors, requiring careful stability analysis (e.g., stiffness constraints on step size Δt).

Example: Solve the non-homogeneous equation:

$$\frac{dy}{dt} = y + t, \quad y(0) = y_0. \tag{1.3}$$

Using an integrating factor $\mu(t) = e^{-t}$, we rewrite the equation as $\frac{d}{dt}(e^{-t}y) = te^{-t}$. Integrating by parts gives the exact analytical solution: $y(t) = (y_0 + 1)e^t - t - 1$.
Comparing this exact benchmark against explicit Euler steps demonstrates how numerical errors accumulate over time and how higher-order Runge-Kutta schemes suppress this error.

1.3 Partial Differential Equations

Partial differential equations relate an unknown function of several independent variables (such as spatial coordinates $\mathbf{x} \in \mathbb{R}^d$ and time t) to its partial derivatives. Such equations are indispensable when physical phenomena depend on local spatial variations and spatial gradients:

$$\frac{\partial u}{\partial t} = f\left(u, \frac{\partial u}{\partial x}, \frac{\partial^2 u}{\partial x^2}\right). \tag{1.4}$$

A classical example is thermal diffusion, described by the **Heat Equation**:

$$\frac{\partial u}{\partial t} = \kappa \frac{\partial^2 u}{\partial x^2}, \tag{1.5}$$

where $u(x,t)$ represents temperature and $\kappa > 0$ is the thermal diffusivity. Depending on the initial and boundary conditions, one observes different behaviors across the spatial domain.

The general form of a partial differential equation is expressed as:

$$F\left(x_i, t, u, \frac{\partial u}{\partial x_i}, \frac{\partial u}{\partial t}, \frac{\partial^2 u}{\partial x_i \partial x_j}, \dots\right) = 0. \tag{1.6}$$

The **order** of a PDE is defined as the order of the highest derivative appearing in it, providing the primary basis for classification:

- **First-order equations**: typically model pure transport and wave propagation phenomena along characteristics.

$$\frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} = 0 \quad \text{(First-order linear transport)}. \tag{1.7}$$

- **Second-order equations**: the most prevalent in classical physics, arising from physical flux balances such as diffusion, viscous stresses, and electrostatics. In the example below, if $\kappa = c^2$, the equations is said to be hyperbolic; otherwise, is parabolic.

$$\frac{\partial u}{\partial t} = \kappa \frac{\partial^2 u}{\partial x^2} \tag{1.8}$$

- **Higher-order equations**: are rarer.

Depending on their order and type, PDEs exhibit distinct qualitative properties that govern the solution behavior prior to computation:

- **First-order transport equations**: Information propagates along characteristic lines at finite speed c. An initial profile $u(x, 0) = u_0(x)$ is translated rigidly as $u(x, t) = u_0(x - ct)$.
- **Parabolic and Elliptic equations** (e.g., heat equation, Poisson equation): Satisfy a **maximum principle**, meaning the solution remains strictly bounded by the boundary and initial data: these equations are smooth and without irregularities and discontinuities.
- **Hyperbolic equations** (e.g., wave equation, compressible Euler equations): Propagate signals at finite wave speeds. They do *not* smooth the solution; discontinuities, wavefronts, and shock waves persist indefinitely without decay.

Properties determine which boundary conditions must be imposed, how the equation must be discretized, and where a neural network is likely to struggle. Sharp fronts, contact discontinuities, and shocks (hyperbolic) are notoriously harder for neural networks to learn than smooth diffusive solutions (parabolic/elliptic), because standard neural architectures suffer from spectral bias and struggle to represent steep gradients.

1.4 Strong vs. Weak Formulations

A **strong (classical) solution** is a function that is differentiable as many times as the differential equation requires and satisfies the PDE at every single point of the domain Ω.
This requirement is exceedingly stringent: it fails as soon as the physical solution exhibits a corner, a kink, a material interface, or a shock discontinuity (as in Burgers' equation).

A **weak (variational) solution** relaxes pointwise constraints. Instead of requiring the PDE to hold pointwise, we multiply the equation by an arbitrary test function v and integrate over the domain, so that the balance holds *"on average"* against all admissible test functions.

Before introducing an example, let's recall that with the notation $H_0^1(0, 1)$ we mean the set of all the possible functions that are square-integrable, have square-integrable weak first derivatives, and vanish at both endpoints (meaning that their value is 0 at the boundaries $[0, 1]$).

Consider the 1D Poisson boundary value problem:

$$-\frac{d^2 u}{dx^2} = f(x) \quad \text{in } (0, 1), \quad u(0) = u(1) = 0. \tag{1.9}$$

Equation 1.9 represents the **strong form**.

To derive the **weak form**, we multiply by a test function $v \in H_0^1(0,1)$ and integrate over $[0,1]$:

$$-\int_0^1 \frac{d^2u}{dx^2} v \, dx = \int_0^1 f v \, dx. \tag{1.10}$$

Integrating by parts transfers one derivative from u onto the test function v:

$$-\left[\frac{du}{dx} v\right]_0^1 + \int_0^1 \frac{du}{dx}\frac{dv}{dx} \, dx = \int_0^1 f v \, dx. \tag{1.11}$$

Since $v \in H_0^1(0,1)$, the test function vanishes identically on the boundary, $v(0) = v(1) = 0$.
The weak formulation thus reads:

$$u \in H_0^1(0,1) \text{ s.t. } \int_0^1 \frac{du}{dx}\frac{dv}{dx} \, dx = \int_0^1 f v \, dx \quad \forall v \in H_0^1(0,1). \tag{1.12}$$

Weak formulations are essential for two primary reasons: they permit physically meaningful solutions with discontinuities or lower regularity (only requiring square-integrable first weak derivatives), and they constitute the mathematical foundation of the **Finite Element Method (FEM)**, where the infinite-dimensional weak form is projected onto finite-dimensional polynomial subspaces.

1.5 Finite Element Method

Finite-Difference (FDM), Finite-Volume (FVM), and Finite-Element Methods (FEM) discretize PDEs into finite-dimensional systems, meaning that unknown functions are replaced by a (finite, but huge) number of values or coefficients governed by ODEs.

Finite-Differences approximate derivatives using grid values; Finite-Volumes enforce integral conservation through cell-boundary fluxes; Finite-Elements approximate solutions in finite-dimensional function spaces, typically using piecewise polynomial bases and a weak formulation.
In one spatial dimension, all the three methods are almost indistinguishable, leading to the identical three-point stencil and tridiagonal linear system; in higher dimensions, differences appear in terms of computational bottlenecks.
We present here the derivation of the FEM for the 1-dimensional Heat Equation.

1.5.1 Derivation of FEM for the 1D Heat Equation

Consider the 1D heat equation on $\Omega = (0,1)$:

$$\frac{\partial u}{\partial t} = \kappa \frac{\partial^2 u}{\partial x^2}. \tag{1.13}$$

Testing against an admissible test function $v(x)$ and integrating over $[0,1]$ yields:

$$\int_0^1 v \frac{\partial u}{\partial t} \, dx - \kappa \int_0^1 v \frac{\partial^2 u}{\partial x^2} \, dx = 0. \tag{1.14}$$

Integrating the second-derivative term by parts:

$$\int_0^1 v\frac{\partial u}{\partial t}\,dx - \kappa\left[v\frac{\partial u}{\partial x}\right]_0^1 + \kappa\int_0^1 \frac{\partial v}{\partial x}\frac{\partial u}{\partial x}\,dx = 0. \tag{1.15}$$

At this stage, the problem remains in the continuum, infinite-dimensional setting, and we want to discretize using an approximate solution, in an N-dimensional subspace, defined by $X_N = \text{span}\{\phi_1(x), \ldots, \phi_N(x)\}$:

$$u(x,t) \approx \sum_{i=1}^{N} u_i(t)\phi_i(x), \tag{1.16}$$

where $\{\phi_i\}_{i=1}^N$ are spatial basis functions (typically piecewise polynomials) and $u_i(t)$ are time-dependent nodal coefficients.

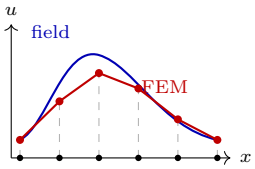


Figure 1.2: FEM represents a field by nodal values joined with piecewise-linear elements.

Following the **Galerkin method**, we select the test functions $v(x)$ from the identical basis, setting $v = \phi_j(x)$ for each $j \in \{1, \ldots, N\}$:

$$\int_0^1 \phi_j\left(\sum_{i=1}^{N}\frac{du_i}{dt}\phi_i(x)\right)dx - \kappa\int_0^1 \phi_j\frac{\partial^2}{\partial x^2}\left(\sum_{i=1}^{N}u_i(t)\phi_i(x)\right)dx = 0. \tag{1.17}$$

Since the integral is w.r.t the spatial dimension x, we can extract the sum with the time-dependent derivatives; applying then integration by parts to the rightmost term we get:

$$\sum_{i=1}^{N}\frac{du_i}{dt}\int_0^1 \phi_j\phi_i\,dx - \kappa\sum_{i=1}^{N}u_i(t)\left[\phi_j\frac{d\phi_i}{dx}\right]_0^1 + \kappa\sum_{i=1}^{N}u_i(t)\int_0^1\frac{d\phi_j}{dx}\frac{d\phi_i}{dx}\,dx = 0, \quad \forall j. \tag{1.18}$$

Assuming $\frac{\partial u}{\partial x}(0) = \frac{\partial u}{\partial x}(1) = 0$ ("homogeneous Neumann boundary conditions"), the central term vanishes:

$$\sum_{i=1}^{N}\frac{du_i}{dt}\int_0^1 \phi_j\phi_i\,dx + \kappa\sum_{i=1}^{N}u_i(t)\int_0^1\frac{d\phi_j}{dx}\frac{d\phi_i}{dx}\,dx = 0, \quad \forall j \tag{1.19}$$

Said **Mass Matrix** M and **Stiffness Matrix** K (both $\in \mathbb{R}^{N\times N}$)

$$M_{ji} = \int_0^1 \phi_j(x)\phi_i(x)\,dx, \tag{1.20}$$

$$K_{ji} = \int_0^1 \frac{d\phi_j}{dx}\frac{d\phi_i}{dx}\,dx, \tag{1.21}$$

we arrive at the linear system of ordinary differential equations:

$$M\frac{d\mathbf{u}}{dt} = -\kappa K\mathbf{u}. \tag{1.22}$$

This FEM step replaces the continuous heat equation with a finite ODE system for nodal values: we gain a computable approximation for the time stepping at the price of an error given by the quality of the mesh!

1.6 Problem Formulation

Let us state the mathematical framework for discussing the general formulation of the FEM applied to parametrized partial differential equations.

Given a parameter vector $\boldsymbol{\mu} \in \mathcal{P} \subset \mathbb{R}^P$, evaluate a scalar output of interest:

$$s^e(\boldsymbol{\mu}) = I(u^e(\boldsymbol{\mu})), \tag{1.23}$$

where the exact field variable $u^e(\boldsymbol{\mu}) \in X^e$ satisfies the parametric variational problem:

$$a(u^e(\boldsymbol{\mu}), v; \boldsymbol{\mu}) = f(v; \boldsymbol{\mu}), \quad \forall v \in X^e. \tag{1.24}$$

The definitions and functional settings are specified as follows:

- $\boldsymbol{\mu}$: input parameter vector (P-tuple) encoding variations in physical properties (e.g., thermal conductivity, Reynolds number, viscosity), source terms, or geometric configurations;
- $\mathcal{P} \subset \mathbb{R}^P$: closed, compact parameter domain;
- $s^e(\boldsymbol{\mu}) \in \mathbb{R}$: scalar output of interest (quantity of interest, QoI), such as aerodynamic drag/lift, average temperature, or surface boundary fluxes;
- $I(\cdot) \in (X^e)'$: linear bounded output functional evaluating the target quantity;
- $u^e(\boldsymbol{\mu})$: exact field variable defined on the physical spatial domain $\Omega \subset \mathbb{R}^d$;
- X^e: an appropriate Hilbert function space over Ω satisfying:

  $$\left(H_0^1(\Omega)\right)^s \subseteq X^e \subseteq \left(H^1(\Omega)\right)^s, \tag{1.25}$$

  where $s \geq 1$ denotes the number of coupled scalar field components (e.g., $s = 1$ for temperature, $s = d$ for velocity);
- $a(\cdot, \cdot; \boldsymbol{\mu}) : X^e \times X^e \to \mathbb{R}$: a parametric bilinear form, assumed continuous and coercive uniformly with respect to $\boldsymbol{\mu}$;
- $f(\cdot; \boldsymbol{\mu}) \in (X^e)'$: continuous linear load functional.

1.7 General FEM formulation

Before reducing a model, we need a reliable numerical model to reduce.
This **high-fidelity**, "truth" model resolves the spatial details of the problem. "Truth" means our numerical reference, not an exact solution: it still has discretization error.
In practice, we consider as our truth method the FEM.

Consider steady heat diffusion along a 1D interval $\Omega = (0, 1)$:

$$-\kappa \frac{\partial^2 u}{\partial x^2} = f(x, \boldsymbol{\mu}), \quad u(0) = u(1) = 0. \tag{1.26}$$

Here u and κ have the usual interpretation, while $f(x, \boldsymbol{\mu})$ describes heat sources controlled by the parameter $\boldsymbol{\mu} \in \mathcal{P}$: changing them produces different temperature profiles.

As in Chapter 1, we use a weak formulation: multiply the equation by a test function $v \in X^e = H_0^1(0, 1)$, integrate, and transfer one derivative from u to v by integrating by parts. We get:

$$\kappa \int_0^1 \frac{\partial v}{\partial x} \frac{\partial u}{\partial x} \, dx = \int_0^1 f(x, \boldsymbol{\mu}) v \, dx, \quad \forall v \in X^e. \tag{1.27}$$

This is a concrete example of Eq. 1.24. Here $f(v;\boldsymbol{\mu})$ denotes the integral load functional, rather than the source function itself.

Since we want to apply FEM, we discretize the interval into a mesh $\mathcal{T}_h$ and approximate the temperature by a continuous, piecewise linear function. The collection of all such functions, satisfying the boundary conditions is

$$X_{\mathcal{N}} = \left\{ v \in X^e : v|_{T_h} \in \mathbb{P}^1(T_h),\, \forall T_h \in \mathcal{T}_h \right\} \subset X^e, \tag{1.28}$$

where T_h is a mesh element, $\mathbb{P}^1(T_h)$ means linear polynomials on that element, and $\dim(X_{\mathcal{N}}) = \mathcal{N}$ counts the independent unknowns.
Applying FEM, we get:

$$u_{\mathcal{N}}(x,\boldsymbol{\mu}) = \sum_{i=1}^{\mathcal{N}} u_i(\boldsymbol{\mu})\phi_i^{\mathrm{FE}}(x). \tag{1.29}$$

The functions describe how to interpolate between nodes; the coefficients $u_i(\boldsymbol{\mu})$ are the unknown nodal temperatures. Computing a profile now means finding $\mathcal{N}$ numbers.

We determine them by requiring the weak balance to hold for every test function defined in the same finite element space:

$$a(u_{\mathcal{N}}(\boldsymbol{\mu}), v; \boldsymbol{\mu}) = f(v;\boldsymbol{\mu}), \quad \forall v \in X_{\mathcal{N}}. \tag{1.30}$$

This is called **Galerkin projection**. Testing with each $v = \phi_j^{\mathrm{FE}}$, $j = 1,\ldots,\mathcal{N}$, gives one equation per unknown:

$$\kappa \sum_{i=1}^{\mathcal{N}} u_i(\boldsymbol{\mu}) \int_0^1 \frac{\partial \phi_j^{\mathrm{FE}}}{\partial x} \frac{\partial \phi_i^{\mathrm{FE}}}{\partial x}\, dx = \int_0^1 f(x,\boldsymbol{\mu})\phi_j^{\mathrm{FE}}\, dx. \tag{1.31}$$

Collecting these equations yields

$$\kappa K_{\mathcal{N}} \mathbf{u}_{\mathcal{N}} = \mathbf{f}_{\mathcal{N}}(\boldsymbol{\mu})$$

Or, generally:

$$A_{\mathcal{N}}(\boldsymbol{\mu})\mathbf{u}_{\mathcal{N}}(\boldsymbol{\mu}) = \mathbf{f}_{\mathcal{N}}(\boldsymbol{\mu}). \tag{1.32}$$

Here $\mathbf{u}_{\mathcal{N}}$ collects the nodal temperatures, $K_{\mathcal{N}}$ encodes their diffusion coupling, and $\mathbf{f}_{\mathcal{N}}$ collects the source contributions; the general matrix $A_{\mathcal{N}}$ allows the operator itself to depend on the parameters.
At this point, we turned the PDE into a finite system for nodal temperatures, so we can compute solutions and outputs by solving the system: the functional I (i.e., the solution of the system) extracts the quantity we actually need:

$$s_{\mathcal{N}}(\boldsymbol{\mu}) = I(u_{\mathcal{N}}(\boldsymbol{\mu})). \tag{1.33}$$

We choose a sufficiently resolved mesh so that $|s^e(\boldsymbol{\mu}) - s_{\mathcal{N}}(\boldsymbol{\mu})| \ll 1$ at the required accuracy. This may require a large $\mathcal{N}$. Repeating these full-order solves for many parameter values motivates reduction.

Linear Reduced-Order Modelling

2

2.1 Introduction

Table of Contents

1. Introduction
2. Reduced-Basis Method
3. Proper Orthogonal Decomposition
4. Greedy Sampling
5. An Example

Many scientific applications require solving the same model repeatedly for different parameters, geometries, or operating conditions. A full-order model (FOM) resolves all $\mathcal{N}$ degrees of freedom, so each solve can be expensive. A reduced-order model (ROM) approximates the solution in a shared low-dimensional space of dimension $N \ll \mathcal{N}$. Instead of solving for every full-order degree of freedom, it evolves a small set of coefficients that describe the dominant solution patterns. When the equations can be projected and assembled efficiently, this can make repeated predictions much faster while retaining useful accuracy.

This chapter develops the main ingredients of projection-based ROMs: the finite-element truth model, reduced-basis approximation, Proper Orthogonal Decomposition (POD), and greedy sampling.
POD is an *explore-and-compress* strategy. First computes full-order solutions, called snapshots, at selected training conditions; then combines them into orthogonal modes ranked by how much of the snapshot variation they capture. For a fixed number of modes, POD minimizes the average projection error over those snapshots (in the chosen inner product) therefore building a compact basis from data already computed.
Greedy sampling takes a different approach: it chooses which full-order solutions to compute next. Starting from a small basis, it evaluates an error estimate over candidate parameters, selects the candidate with the largest estimated error, and adds its truth solution to the basis.

The resulting workflow follows the **offline-online paradigm**. With a FOM of $\mathcal{N}$ degrees of freedom and a ROM of dimension $N \ll \mathcal{N}$:

- **Offline Step**: Sample the parameter domain, compute high-fidelity snapshots, construct a reduced basis, and precompute projected operators. This preparation may be expensive and depend on $\mathcal{N}$.
- **Online Step**: For a new parameter $\boldsymbol{\mu} \in \mathcal{P}$, solve the reduced system and evaluate

$$\boldsymbol{\mu} \longmapsto s_N(\boldsymbol{\mu}) \approx s_{\mathcal{N}}(\boldsymbol{\mu}) \approx s^e(\boldsymbol{\mu}). \tag{2.1}$$

  With an efficient offline-online decomposition, evaluation costs depend primarily on N, not $\mathcal{N}$, enabling rapid repeated queries.

A small reduced dimension alone does not guarantee speed or accuracy: stability, error estimation, parameter coverage, and efficient operator evaluation remain essential.

2.2 Reduced-Basis Method

This section introduces a type of ROM called **projection-based reduced-basis** method.

The core idea of a reduced-basis ROM is to replace the full-order solve with a small system whose basis captures the dominant solution behavior. This system is obtained by projecting the full-order equations onto a smaller dimensional space.
In an offline stage, we solve the FOM at selected parameter values. These solutions, called **snapshots**, form a **reduced basis**: a small set of representative fields whose linear combinations approximate other solutions. For a new parameter, we represent the unknown field as a combination of these basis functions.
The method is efficient when this reduced system can be assembled without repeating full-order work, and its accuracy depends on how well the snapshots capture solutions across the parameter range.

We explain the method using the same notation as the problem formulation in Sec. 1.6. In order to apply the PDE, we choose a value of the model parameters $\boldsymbol{\mu}$ and solve the physical problem. The result is one field $u^e(\boldsymbol{\mu})$. Letting the parameters range over $\mathcal{P}$ gives a family of physically possible solutions, called the **solution manifold**:

$$\mathcal{M} = \left\{ u^e(\boldsymbol{\mu}) \in X^e : \boldsymbol{\mu} \in \mathcal{P} \right\} \subset X^e. \tag{2.2}$$

The solution of the truth method lies instead in the **truth manifold**:

$$\mathcal{M}_\mathcal{N} = \left\{ u_\mathcal{N}(\boldsymbol{\mu}) \in X_\mathcal{N} : \boldsymbol{\mu} \in \mathcal{P} \right\} \subset X_\mathcal{N}. \tag{2.3}$$

Each truth solution has $\mathcal{N}$ coefficients, which are not independently: maybe, if one changes a parameter $\mu_1 \in \boldsymbol{\mu}$, then also μ_2 changes as well!

A ROM asks whether these profiles can be represented accurately using only $N \ll \mathcal{N}$ shared patterns.
The **Kolmogorov N-width** describes the (best) worst-case accuracy any N-dimensional linear space could achieve:

$$d_N(\mathcal{M})_X = \inf_{V_N \subset X^e} \sup_{u \in \mathcal{M}} \inf_{w \in V_N} \|u - w\|_X. \tag{2.4}$$

where $\dim(V_N) = N$.
In practical terms, small d_N means some N patterns can approximate every solution across the parameter range; slow decay means many patterns are needed.
The obtained width is an ideal limit, not a guarantee that a particular snapshot method finds the best space!

The following steps build a practical basis from selected truth solutions (snapshots) and **project** the governing equations onto this **small-dimensional manifold**.

Step 1: Collect representative solutions.
Choose N parameter points in $\mathcal{P}$:

$$S_N = \{\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \ldots, \boldsymbol{\mu}_N\} \subset \mathcal{P}, \tag{2.5}$$

At each parameter choice, solve the full-order model once. A snapshot is the resulting temperature profile: it stores one value at each of the $\mathcal{N}$ mesh

nodes. Linear combinations of these snapshots form the **Reduced Basis space**:

$$W_N = \text{span}\{\zeta_n = u_{\mathcal{N}}(\boldsymbol{\mu}_n),\ 1 \le n \le N\}, \tag{2.6}$$

Assume the selected snapshots are linearly independent (so the space dimension is N); if they are not, just orthonormalize them and discard redundant snapshots.
Choosing representative parameter points is crucial: snapshots must capture the behavior expected at later queries.

Step 2: Enforce the physics in the smaller space.
Given $\boldsymbol{\mu} \in \mathcal{P}$, we evaluate $s_N(\boldsymbol{\mu}) = I(u_N(\boldsymbol{\mu}))$.
We require $u_N(\boldsymbol{\mu}) \in W_N$ to satisfy the same weak balance as the truth model, but only against test functions in W_N:

$$a(u_N(\boldsymbol{\mu}), v; \boldsymbol{\mu}) = f(v; \boldsymbol{\mu}), \quad \forall v \in W_N. \tag{2.7}$$

This is Galerkin projection onto the reduced space.

At this point, we have a reduced model built from N patterns, which keeps the weak physical balance within this space and reduces the number of unknowns from $\mathcal{N}$ to N.
What we do not yet know are the N coefficients that tell us how to combine the patterns for a given parameter value; to find these coeffcients, in the next step we form and solve a system of N equations.
Details not captured by the chosen coefficients patterns will be lost.

Step 3: Solve for the mixing coefficients.
Using the snapshot basis $\{\zeta_j\}_{j=1}^N$, we can write the unknown profile as:

$$u_N(\boldsymbol{\mu}) = \sum_{j=1}^{N} u_{N,j}(\boldsymbol{\mu})\zeta_j. \tag{2.8}$$

The coefficients $u_{N,j}(\boldsymbol{\mu})$ tell us how much of each stored pattern to use.
Testing Equation (2.7) against each $v = \zeta_i$, $i = 1, \ldots, N$, gives N equations for these N unknowns:

$$\sum_{j=1}^{N} a(\zeta_j, \zeta_i; \boldsymbol{\mu}) u_{N,j}(\boldsymbol{\mu}) = f(\zeta_i; \boldsymbol{\mu}), \quad 1 \le i \le N. \tag{2.9}$$

Once $\mathbf{u}_N(\boldsymbol{\mu}) = [u_{N,1}, \ldots, u_{N,N}]^T$ is known, if I is linear we can compute the output without reconstructing the entire spatial field:

$$s_N(\boldsymbol{\mu}) = \sum_{j=1}^{N} u_{N,j}(\boldsymbol{\mu}) I(\zeta_j). \tag{2.10}$$

The values $I(\zeta_j)$ can be stored offline, and the online output evaluation is then just a weighted sum!
We thus have equations for the N coefficients and a way to compute the output. Still, we need to connected these equations to the FEM matrices; we will use the snapshot matrix Z to form the reduced system.

Step 4: Connect the reduced basis to the existing solver.
This step is a bit mathematical, please don't be scared!
Each snapshot is already represented on the full-order mesh, so it has an expansion in the finite element basis:

$$\zeta_j(x) = \sum_{k=1}^{\mathcal{N}} Z_{kj} \phi_k^{\text{FE}}(x), \tag{2.11}$$

The **snapshot matrix** $Z \in \mathbb{R}^{\mathcal{N}\times N}$ stores these coefficients: column j contains the $\mathcal{N}$ nodal values of snapshot j:

$$Z = \left[\mathbf{u}_{\mathcal{N}}(\boldsymbol{\mu}_1) \mid \mathbf{u}_{\mathcal{N}}(\boldsymbol{\mu}_2) \mid \ldots \mid \mathbf{u}_{\mathcal{N}}(\boldsymbol{\mu}_N)\right] \in \mathbb{R}^{\mathcal{N}\times N}. \tag{2.12}$$

Multiplying the N reduced coefficients by Z gives the approximate solution at all $\mathcal{N}$ mesh nodes. Multiplying the full-order equations by Z^T enforces the balance along the snapshot directions. Together, these steps give the reduced matrix $Z^T A_{\mathcal{N}} Z$. The equations below show this directly:

$$\begin{aligned} a(\zeta_j, \zeta_i) &= a\left(\sum_{k=1}^{\mathcal{N}} Z_{kj}\phi_k^{\mathrm{FE}}, \sum_{l=1}^{\mathcal{N}} Z_{li}\phi_l^{\mathrm{FE}}\right) \\ &= \sum_{k=1}^{\mathcal{N}}\sum_{l=1}^{\mathcal{N}} Z_{li}\, a(\phi_k^{\mathrm{FE}}, \phi_l^{\mathrm{FE}}) Z_{kj} \\ &= \sum_{l=1}^{\mathcal{N}} Z_{li}\left(A_{\mathcal{N}} Z\right)_{lj} \\ &= \left(Z^T A_{\mathcal{N}} Z\right)_{ij}. \end{aligned} \tag{2.13}$$

For readibility, we have omitted the parameter $\boldsymbol{\mu}$.

The load is tested against the same patterns, giving the corresponding projected right-hand side:

$$\begin{aligned} f(\zeta_i) &= f\left(\sum_{l=1}^{\mathcal{N}} Z_{li}\phi_l^{\mathrm{FE}}\right) \\ &= \sum_{l=1}^{\mathcal{N}} Z_{li} f(\phi_l^{\mathrm{FE}}) \\ &= \left(Z^T \mathbf{f}_{\mathcal{N}}\right)_i. \end{aligned} \tag{2.14}$$

Applying these operations to the full-order system (Eq. 1.32) gives:

$$\left(Z^T A_{\mathcal{N}}(\boldsymbol{\mu}) Z\right)\mathbf{u}_N(\boldsymbol{\mu}) = Z^T \mathbf{f}_{\mathcal{N}}(\boldsymbol{\mu}), \tag{2.15}$$

Instead of solving for $\mathcal{N}$ nodal values, we solve the compact, generally dense $N \times N$ system:

$$A_N(\boldsymbol{\mu})\mathbf{u}_N(\boldsymbol{\mu}) = \mathbf{f}_N(\boldsymbol{\mu}), \tag{2.16}$$

Its matrix and load are precisely the projected full-order quantities:

$$A_N(\boldsymbol{\mu}) = Z^T A_{\mathcal{N}}(\boldsymbol{\mu}) Z \in \mathbb{R}^{N\times N}, \quad \mathbf{f}_N(\boldsymbol{\mu}) = Z^T \mathbf{f}_{\mathcal{N}}(\boldsymbol{\mu}) \in \mathbb{R}^N. \tag{2.17}$$

For the 1D diffusion example, the same construction gives $\kappa K_N \mathbf{u}_N = \mathbf{f}_N$ with $K_N = Z^T K_{\mathcal{N}} Z$. The physical diffusion law is unchanged; only the space used to represent its solution is smaller.

The main computational costs are:

- **Offline**: Compute N full-order snapshots and project the operators. For sparse operators with a fixed number of components and bounded nonzeros per row, the cost is $\mathcal{O}(N\,\mathrm{Cost(FOM)} + \mathcal{N}N^2)$. Store the reduced operators and output weights for reuse.
- **Online**: Solve the dense $N \times N$ reduced system, at cost $\mathcal{O}(N^3)$, then evaluate the output in $\mathcal{O}(N)$. This avoids dependence on $\mathcal{N}$ only if the reduced system can be assembled without revisiting the full mesh.

Step 5: Assemble reduced operators
To do this, write the operator and load as sums of fixed components multiplied by weights that depend on $\boldsymbol{\mu}$:

$$a(u, v; \boldsymbol{\mu}) = \sum_{q=1}^{Q_a} \theta_q^a(\boldsymbol{\mu}) a_q(u, v), \quad f(v; \boldsymbol{\mu}) = \sum_{q=1}^{Q_f} \theta_q^f(\boldsymbol{\mu}) f_q(v). \tag{2.18}$$

This is called **affine parameter dependence**: the components a_q and f_q are fixed, while their weights change with the parameter.

Offline, project and store each component: $A_{N,q} = Z^T A_{\mathcal{N},q} Z$, along with the reduced load components. Online, combine them using the parameter-dependent weights.
Assembling the reduced matrix costs $\mathcal{O}(Q_a N^2)$; the load is assembled in the same way. If the number of components stays fixed and the weights can be evaluated from $\boldsymbol{\mu}$ alone, this step does not require the full mesh.

In short, compute snapshots and store projected components offline; then assemble and solve the small system and evaluate the output online. This separation enables the speedup.

2.3 Proper Orthogonal Decomposition

The snapshot basis in the previous section can contain repeated information: several solutions may have very similar shapes.
Proper Orthogonal Decomposition (POD) combines these snapshots into a smaller set of orthogonal spatial patterns, called **modes**. Rather than keeping particular solutions, it keeps the patterns that best represent the training collection. This gives an *explore-and-compress* strategy:

1. **Explore**: Choose representative parameters $\Xi = \{\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_n\} \subset \mathcal{P}$;
2. **Collect**: Compute the truth solution $u_{\mathcal{N}}(\boldsymbol{\mu})$ at every training point;
3. **Compress**: Find the shared patterns and retain as many as the desired accuracy requires.

where Ξ is composed by n tranining parameter values, .
We can see POD as a mass-weighted and uncentered PCA decomposition!

Among the N-dimensional spaces spanned by the training snapshots, POD chooses the one with the smallest average projection error:

$$X_{N,\mathrm{POD}}^{\mathcal{N}} = \arg \inf_{\dim(X_N)=N} \|u_{\mathcal{N}} - \Pi_{X_N} u_{\mathcal{N}}\|_{L^2(\Xi;X)}, \tag{2.19}$$

Here Π_{X_N} selects the closest approximation in X_N, measured with the spatial norm of X. Equal weights give the average error over the training snapshots; this does not guarantee accuracy at every parameter value.

When there are far fewer snapshots than mesh unknowns ($n \ll \mathcal{N}$), the **Method of Snapshots** computes the POD modes through a smaller eigenvalue problem: it compares snapshots in pairs instead of solving a problem sized by the full mesh. We present here its main steps.

Step 1: Measure similarity between snapshots. Put the n full-order snapshots into the columns of $Z \in \mathbb{R}^{\mathcal{N}\times n}$. Their pairwise similarities form the correlation matrix $\mathbb{C} \in \mathbb{R}^{n\times n}$:

$$\mathbb{C}_{ij} = \frac{1}{n}\big(u_{\mathcal{N}}(\boldsymbol{\mu}_i^{\text{train}}), u_{\mathcal{N}}(\boldsymbol{\mu}_j^{\text{train}})\big)_X, \quad 1 \le i, j \le n, \tag{2.20}$$

The inner product $(\cdot,\cdot)_X$ measures snapshot size and overlap, and determines which features matter. The snapshots are computed offline; this matrix is built from them.

Step 2: Rank the shared patterns. Solve:

$$\mathbb{C}\boldsymbol{\psi}_k = \lambda_k \boldsymbol{\psi}_k, \quad 1 \le k \le n, \tag{2.21}$$

Each eigenpair ($\boldsymbol{\psi}_k \in \mathbb{R}^n, \lambda_k \in \mathbb{R}_0^+$) has two roles: the eigenvector specifies a combination of snapshots, and its eigenvalue measures the average squared content captured by the corresponding mode. Choose Euclidean-orthonormal eigenvectors and order the eigenvalues: $\lambda_1 \ge \lambda_2 \ge \cdots \ge \lambda_n \ge 0$.

Step 3: Recover spatial modes. Eigenvectors contain weights, not temperature profiles. Combine the original snapshots using those weights to recover the functions $\{\zeta_n\}_{n=1}^N$:

$$\zeta_n = \frac{1}{\sqrt{n\lambda_n}} \sum_{i=1}^{n} (\boldsymbol{\psi}_n)_i u_{\mathcal{N}}(\boldsymbol{\mu}_i^{\text{train}}), \quad 1 \le n \le N, \tag{2.22}$$

The denominator normalizes each mode, giving $(\zeta_n, \zeta_m)_X = \delta_{nm}$: each has unit norm and distinct modes are orthogonal. Retain only modes with $\lambda_n > 0$, since zero eigenvalues contain no additional information and cannot be used in this normalization. Their combinations form the POD space:

$$X_{N,\text{POD}}^{\mathcal{N}} = \operatorname{span}\{\zeta_1, \ldots, \zeta_N\}. \tag{2.23}$$

Step 4: Decide how many modes to keep. Each omitted mode contributes its eigenvalue to the average squared projection error. With the equally weighted training norm defined above, POD satisfies the exact identity:

$$\|u_{\mathcal{N}} - \Pi_{X_{N,\text{POD}}^{\mathcal{N}}} u_{\mathcal{N}}\|_{L^2(\Xi;X)}^2 = \sum_{k=N+1}^{n} \lambda_k. \tag{2.24}$$

Equation (2.24) makes the compression trade-off explicit: keeping more modes reduces the discarded sum. Fast eigenvalue decay means that few patterns capture most of the training collection. No other N-dimensional linear space gives a smaller training projection error.

To choose the retained dimension $N_{\max}$, divide discarded squared content by the total and take a square root, and pick the smallest N satisfying:

$$\varepsilon_N^{\text{POD}} = \sqrt{\frac{\sum_{k=N+1}^{n} \lambda_k}{\sum_{k=1}^{n} \lambda_k}} \le \varepsilon_{\text{tol}}, \tag{2.25}$$

This ratio controls relative root-mean-square *projection* error on the training set: while POD constructs the basis, the projected equations still determine the online coefficients.

2.4 Greedy Sampling

POD first computes a collection of truth solutions, then finds a small set of patterns that best represents their average behavior.
Greedy sampling takes a different approach: it starts with a small basis and adds a truth solution where the current model's estimated error is largest. In short, POD compresses solutions already computed; greedy sampling chooses which new solutions to compute and focuses on poorly represented cases.
The candidate set $\Xi \subset \mathcal{P}$ need not have precomputed truth solutions in greedy sampling: error estimates guide the selection.

Le'ts introduce here an ideal, greedy algorithm. Later on, we will present a less expensive approach based on a technique called **A-Posteriori Error Bounds**.
To present the ideal algorithm, we need tat first to imagine tha the true error is available: pick $\boldsymbol{\mu}_1 \in \Xi$, compute its snapshot, and initialize the matrices $S_1 = \{\boldsymbol{\mu}_1\}$ and $W_1 = \text{span}\{u_{\mathcal{N}}(\boldsymbol{\mu}_1)\}$.
For $N = 2, \ldots, N_{\max}$, choose the candidate where the current reduced solution differs most from the truth:

$$\boldsymbol{\mu}_N = \arg\max_{\boldsymbol{\mu}\in\Xi} \|u_{\mathcal{N}}(\boldsymbol{\mu}) - u_{N-1}(\boldsymbol{\mu})\|_X, \tag{2.26}$$

Add this point to $S_N = S_{N-1} \cup \{\boldsymbol{\mu}_N\}$ and enrich the space:

$$W_N = W_{N-1} \oplus \text{span}\big\{u_{\mathcal{N}}(\boldsymbol{\mu}_N)\big\}. \tag{2.27}$$

The symbol $\oplus$ assumes that the snapshot adds an independent direction, i.e., practically, we need to orthogonalize it against the existing basis and discard a redundant snapshot.
Each enrichment adds some behavior that the current model misses.

The approach is here only *ideal* because we need to know the truth at every candidate in order to compute Eq. 2.26, meaning n solves per iteration!

2.4.1 A-Posteriori Error Bounds Sampling

The core idea is pretty similar to the greedy algorithm presented above.
However, here, an **a-posteriori error bound** $\Delta_N(\boldsymbol{\mu})$ is computed after obtaining the reduced solution. Instead of the exact error, it supplies a guaranteed upper bound under the model's stability assumptions:

$$\|u_{\mathcal{N}}(\boldsymbol{\mu}) - u_N(\boldsymbol{\mu})\|_X \leq \Delta_N(\boldsymbol{\mu}), \quad \forall \boldsymbol{\mu} \in \mathcal{P}. \tag{2.28}$$

To obtain a valid and efficient Δ_N we need to introduce the **residual** $r_N(v;\boldsymbol{\mu}) = f(v;\boldsymbol{\mu}) - a(u_N(\boldsymbol{\mu}), v;\boldsymbol{\mu})$, whose norm is the largest imbalance left in the full weak equations.

Keep in mind that a small imbalance does not always mean a small solution error: an unstable problem can amplify it.
For the problems considered here, we can avoid this by simply dividing the norm of the residual by a positive lower bound $\alpha_{\text{LB}}(\boldsymbol{\mu})$ on the stability:

$$\Delta_N(\boldsymbol{\mu}) = \frac{\|r_N(\cdot;\boldsymbol{\mu})\|_{X'}}{\alpha_{\text{LB}}(\boldsymbol{\mu})}. \tag{2.29}$$

In practice:

1. Compute one initial truth solution: $S_1 = \{\boldsymbol{\mu}_1\}$, $W_1 = \text{span}\{u_{\mathcal{N}}(\boldsymbol{\mu}_1)\}$.
2. For $N = 2, \dots, N_{\max}$, evaluate the current reduced model and bound across the candidates, $\Delta_N(\boldsymbol{\mu})$.
3. Select the largest bound, compute a truth solution only there, and enrich:

$$\boldsymbol{\mu}_N = \arg\max_{\boldsymbol{\mu}\in\Xi} \Delta_{N-1}(\boldsymbol{\mu}), \tag{2.30}$$

$$S_N = S_{N-1} \cup \{\boldsymbol{\mu}_N\}, \tag{2.31}$$

$$W_N = W_{N-1} \oplus \text{span}\big\{u_{\mathcal{N}}(\boldsymbol{\mu}_N)\big\}. \tag{2.32}$$

4. Stop when $\max_{\boldsymbol{\mu}\in\Xi} \Delta_N(\boldsymbol{\mu}) \leq \varepsilon_{\text{tol}}$, checking also the initial basis, or report that $N_{\max}$ was reached before meeting the tolerance.

The theoretical convergence analysis of the algorithm is provided in the slides, and substantially is based on Kolmogorov widths: if the best N-dimensional spaces approximate the solution family with algebraically decreasing error, the greedy spaces also converge algebraically.
This obviously requires a reliable, sufficiently sharp estimator and a good candidate set.

2.5 An Example

The following is an example of the framework developed in this chapter, and is based on the so-called **Advection-Diffusion** equation.

The one-dimensional model is

$$\frac{\partial u}{\partial t} + \frac{\partial u}{\partial x} = \frac{1}{\text{Pe}} \frac{\partial^2 u}{\partial x^2}, \tag{2.33}$$

where $u(x,t)$ is the transported field and the *Peclet number* Pe compares advection with diffusion.
Larger Pe means weaker diffusion relative to transport; we will compare $\text{Pe} = 10, 100, 1000$; for each fixed value, selected times along the truth trajectory form the snapshot set.
Time is here the evolution variable, not more an independent parameter.

Weak Formulation As usual, we multiply Eq. 2.33 by a test function v and integrate over $x \in (0,1)$ to get the weak formulation.
Applying integration by parts, and considering the diffusion term to vanish at the boundary, we get:

$$\int_0^1 \frac{\partial u}{\partial t} v \, dx + \int_0^1 \frac{\partial u}{\partial x} v \, dx = -\frac{1}{\text{Pe}} \int_0^1 \frac{\partial u}{\partial x} \frac{\partial v}{\partial x} \, dx. \tag{2.34}$$

FEM Discretization We divide the interval into finite elements and approximate the solution using $\mathcal{N}$ basis functions:

$$u(x,t) = \sum_{i=1}^{\mathcal{N}} u_i(t) \phi_i(x). \tag{2.35}$$

Testing with each finite-element basis function ϕ_j gives a system of ordinary differential equations:

$$M\frac{d\mathbf{u}}{dt} + C\mathbf{u} = -\frac{1}{\mathrm{Pe}}K\mathbf{u}. \tag{2.36}$$

Where the mass, advection, and diffusion matrices are:

$$M_{ij} = \int_0^1 \phi_i \phi_j \, dx, \tag{2.37}$$

$$C_{ji} = \int_0^1 \frac{\partial \phi_i}{\partial x} \phi_j \, dx, \tag{2.38}$$

$$K_{ij} = \int_0^1 \frac{\partial \phi_i}{\partial x} \frac{\partial \phi_j}{\partial x} \, dx. \tag{2.39}$$

The mass matrix couples the time derivative to the finite-element coefficients; this is the time-dependent counterpart of the finite-element truth discretization in Equation (1.30): both expand the solution in a finite-element basis and test against that basis, while the time derivative makes the resulting system an ODE.
The hat-shaped basis functions are local: each is nonzero only around its associated mesh node.

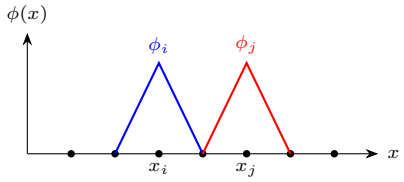


Figure 2.1: Local linear basis functions.

Time Stepping and Reference Solutions The specify a mesh with $n_{\mathrm{el}} = 1000$, a time horizon $T = 0.5$, a time step $\Delta t = 10^{-6}$, and Peclet numbers $\mathrm{Pe} \in \{10, 100, 1000\}$. The initial condition is a narrow Gaussian pulse centered at $x_0 = 0.2$ with width parameter $w_0 = 0.01$:

$$u(x, 0) = \exp\left[-\left(\frac{x - x_0}{w_0}\right)^2\right]. \tag{2.40}$$

For each Peclet number, the full-order model advances this profile in time; we use the semi-discrete operator

$$A = \frac{1}{\mathrm{Pe}}K + C, \tag{2.41}$$

and the Euler update

$$\mathbf{u}^{n+1} = \mathbf{u}^n - \Delta t \, M^{-1} A \mathbf{u}^n. \tag{2.42}$$

Saving the resulting solutions at selected times produces the **snapshot matrix** used to build the POD basis. Figure 2.2 shows the pulse moving right as diffusion spreads and lowers it; at larger Pe, it remains narrower.

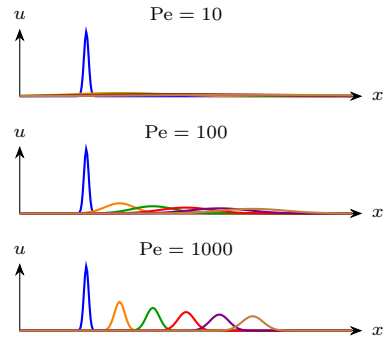


Figure 2.2: Snapshots from $t = 0$ to $t = 0.5$ in steps of 0.1 (blue to brown).

Snapshots and POD Basis We treat the saved full-order profiles as training data.
As we have seen, POD compresses them by finding orthogonal spatial modes that represent the snapshots well in the finite-element inner product.
Let S denote the matrix having snapshots as columns and let M be the finite-element mass matrix. We compute the correlation matrix $\mathbb{C} = S^T M S$ from the snapshots and solve its eigenvalue problem: $\mathbb{C}\boldsymbol{\psi}_k = \lambda_k \boldsymbol{\psi}_k$. We then order the eigenvalues as $\lambda_1 \geq \lambda_2 \geq \cdots \geq 0$; their associated eigenvectors combine the snapshots into POD modes. The, we collect the first N normalized modes as columns of Z: this is the matrix that changes coordinates from the finite-element basis to the reduced basis.

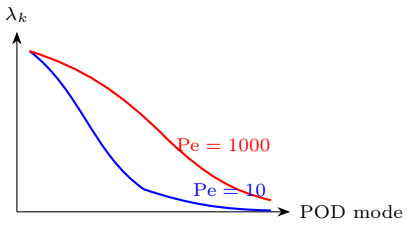


Figure 2.3: Eigenvalues' decays.

The number of retained modes is selected by a cumulative eigenvalue threshold, following Eq. 2.25. This criterion measures how well the modes represent the training snapshots; by itself it does not guarantee the accuracy of a time-evolved ROM.

Fig. 2.3 show slower eigenvalue decay for Pe = 1000 than for Pe = 10, indicating that the high-Peclet snapshots require more modes for comparable representation.

Projected ROM The full-order solution is approximated in the POD basis by $\mathbf{u} \approx Z\mathbf{u}_N$, where $\mathbf{u}_N$ contains only N reduced coefficients.
Projecting the finite-element time-stepping equation onto this basis gives

$$(Z^T M Z)\frac{\mathbf{u}_N^{n+1} - \mathbf{u}_N^n}{\Delta t} + (Z^T C Z)\mathbf{u}_N^n = -\frac{1}{\mathrm{Pe}}(Z^T K Z)\mathbf{u}_N^n. \tag{2.43}$$

Multiplying the full-order residual by Z^T tests it against the retained modes, exactly the Galerkin idea in Equation (2.7).
The projected mass, advection, and diffusion matrices are $Z^T M Z$, $Z^T C Z$, and $Z^T K Z$.
Once assembled, each time step evolves N reduced coefficients rather than $\mathcal{N}$ finite-element values; reconstruct a full profile as $Z\mathbf{u}_N$ when needed.

This example is an **intrusive ROM**: it projects the governing finite-element equations; the online workflow is efficient only when the reduced operators can be assembled without repeating full-order work.
The advection-diffusion model we are considering has fixed matrices for a given Pe, making these projections reusable through the time integration.

2.5.1 Nonlinear Advection

This extension replaces the linear advection term in the example with a nonlinear one, as we will see for the Burgers model of Equation (3.1):

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \frac{1}{\mathrm{Pe}}\frac{\partial^2 u}{\partial x^2}. \tag{2.44}$$

After finite-element and time discretization, the convective matrix depends on the current solution:

$$M\frac{\mathbf{u}^{t+1} - \mathbf{u}^t}{\Delta t} + C(\mathbf{u}^t)\mathbf{u}^t = -\frac{1}{\mathrm{Pe}}K\mathbf{u}^t. \tag{2.45}$$

Consequently, $C(\mathbf{u}^t)$ must be assembled at every time step. Projecting gives the reduced evolution equation

$$(Z^T M Z)\frac{\mathbf{u}_N^{t+1} - \mathbf{u}_N^t}{\Delta t} + (Z^T C(Z\mathbf{u}_N^t)Z)\mathbf{u}_N^t = -\frac{1}{\mathrm{Pe}}(Z^T K Z)\mathbf{u}_N^t. \tag{2.46}$$

This is the same bottleneck derived in Equation (3.7): although $\mathbf{u}_N$ has only N entries, naive evaluation still requires lifting it to the full-order space and evaluating the nonlinear operator across the mesh. Chapter 3, particularly Section 3.3, introduces hyper-reduction to avoid this full-order online work.
The reported high-Peclet timing illustrates the cost of the naive evaluation; the speed-up depends on implementation and is not determined by reduced dimension alone.

Nonlinear Reduced-Order Models

3

3.1 Introduction

Table of Contents

1. Nonlinear Problems
2. Discrete Empirical Interpolation Method
3. Reduced Models for Navier–Stokes
4. Non-Intrusive ROMs
5. Error and Validation

Chapter 2 introduced projection-based reduced-order models. This chapter explains why nonlinear terms can limit their online efficiency and how DEIM reduces that cost. It also covers reduced models for incompressible flow and non-intrusive ROMs, which learn from simulation data.

3.2 Nonlinear Problems

Nonlinear dynamics can produce sharp fronts, multiple states, or turbulent flow, making them challenging to approximate and validate. Even when a reduced basis captures the state accurately, nonlinear terms can prevent the resulting model from being computationally inexpensive. The viscous Burgers equation provides a simple example of this distinction between reducing the state dimension and reducing the cost of evaluating the equations.

Consider the time-dependent viscous Burgers equation on $\Omega = (0, 1)$:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \kappa \frac{\partial^2 u}{\partial x^2} + f(x, \boldsymbol{\mu}). \tag{3.1}$$

Testing with $v \in H_0^1(0, 1)$ and integrating the diffusion term by parts gives

$$\int_0^1 \frac{\partial u}{\partial t} v \, dx + \int_0^1 u\frac{\partial u}{\partial x} v \, dx = -\kappa \int_0^1 \frac{\partial u}{\partial x}\frac{\partial v}{\partial x} \, dx + \int_0^1 f(x, \boldsymbol{\mu}) v \, dx. \tag{3.2}$$

Expand $u(x, t) \approx \sum_{i=1}^{\mathcal{N}} u_i(t)\phi_i(x)$ and test with $v = \phi_j(x)$ for $j = 1, \ldots, \mathcal{N}$:

$$\begin{aligned}\sum_{i=1}^{\mathcal{N}} \frac{\partial u_i}{\partial t} \int_0^1 \phi_i \phi_j \, dx + \int_0^1 \left(\sum_{k=1}^{\mathcal{N}} u_k \phi_k\right) \left(\sum_{i=1}^{\mathcal{N}} u_i \frac{\partial \phi_i}{\partial x}\right) \phi_j \, dx \\ = -\kappa \sum_{i=1}^{\mathcal{N}} u_i \int_0^1 \frac{\partial \phi_i}{\partial x}\frac{\partial \phi_j}{\partial x} \, dx + \int_0^1 f(x, \boldsymbol{\mu}) \phi_j \, dx.\end{aligned} \tag{3.3}$$

In matrix form, the finite-element discretization is a nonlinear system of ODEs:

$$M\frac{d\mathbf{u}}{dt} + C(\mathbf{u})\mathbf{u} = -K\mathbf{u} + \mathbf{f}, \tag{3.4}$$

where M and K are the mass and stiffness matrices, and $C(\mathbf{u})$ is the state-dependent convective matrix:

$$C(\mathbf{u})_{ji} = \sum_{k=1}^{\mathcal{N}} u_k \int_0^1 \phi_k \frac{\partial \phi_i}{\partial x} \phi_j \, dx. \tag{3.5}$$

With an explicit Euler time step, this becomes

$$M\frac{\mathbf{u}^{t+1}-\mathbf{u}^t}{\Delta t}+C(\mathbf{u}^t)\mathbf{u}^t=-K\mathbf{u}^t+\mathbf{f}. \tag{3.6}$$

Now approximate the finite-element state using a reduced basis $Z\in\mathbb{R}^{\mathcal{N}\times N}$, with $\mathbf{u}\approx Z\mathbf{u}_N$. Galerkin projection yields

$$\big(Z^TMZ\big)\frac{\mathbf{u}_N^{t+1}-\mathbf{u}_N^t}{\Delta t}+Z^TC(Z\mathbf{u}_N^t)Z\mathbf{u}_N^t=-\big(Z^TKZ\big)\mathbf{u}_N^t+Z^T\mathbf{f}. \tag{3.7}$$

Although the state now has only N coefficients, evaluating its nonlinear contribution,

$$\mathbf{g}_N(\mathbf{u}_N)=Z^TC(Z\mathbf{u}_N)Z\mathbf{u}_N, \tag{3.8}$$

typically requires reconstructing $\mathbf{u}=Z\mathbf{u}_N$, evaluating the convective operator over all $\mathcal{N}$ degrees of freedom, and projecting the result with Z^T. Thus, standard projection alone does not guarantee an online cost that depends only on N; this full-order nonlinear evaluation can dominate the reduced solve.

This bottleneck motivates **hyper-reduction**.
The next section develops the a tool for doing such hyper-reduction: **Discrete Empirical Interpolation Method (DEIM)**. DEIM builds a basis for snapshots of the nonlinear residual and selects a small set of entries at which to evaluate it. The selected residual values are then mapped back into the reduced equations, avoiding a full-mesh nonlinear evaluation when the operator permits local sampling. This differs from a **non-intrusive ROM**, which learns the reduced dynamics from data rather than projecting the governing equations; non-intrusive approaches are discussed in Section 3.5.

3.3 Discrete Empirical Interpolation Method

As shown in Section 3.2 and illustrated by the nonlinear advection example in Section 2.5.1, the reduced state may have only N coefficients while evaluating the projected convective term in Equation (3.7) still requires full-order work. The **Discrete Empirical Interpolation Method (DEIM)** addresses this cost by approximating the nonlinear term in a small spatial basis and evaluating it only at selected entries.

Let $\mathbf{f}_{\mathrm{nl}}(\mathbf{u})\in\mathbb{R}^{\mathcal{N}}$ denote the full-order nonlinear residual. From representative full-order solutions, collect nonlinear snapshots

$$F_{\mathrm{nl}}=\big[\mathbf{f}_{\mathrm{nl}}(\mathbf{u}^1),\dots,\mathbf{f}_{\mathrm{nl}}(\mathbf{u}^{n_{\mathrm{snap}}})\big]. \tag{3.9}$$

Applying POD to these residual snapshots gives a nonlinear basis $G_m\in\mathbb{R}^{\mathcal{N}\times m}$. A selection matrix $P_m\in\mathbb{R}^{\mathcal{N}\times m}$ extracts m entries of a vector. The DEIM approximation is

$$\mathbf{f}_{\mathrm{nl}}(\mathbf{u})\approx G_m(P_m^TG_m)^{-1}P_m^T\mathbf{f}_{\mathrm{nl}}(\mathbf{u}), \tag{3.10}$$

where the selected indices are chosen so that $P_m^TG_m$ is invertible and well-conditioned. The matrix P_m selects spatial entries; it is distinct from the parameter-domain notation $\mathcal{P}$ used in Chapter 2.

Projecting Equation (3.10) onto the state basis Z gives

$$Z^T\mathbf{f}_{\mathrm{nl}}(Z\mathbf{a})\approx B_m\,\mathbf{f}_p(\mathbf{a}),\qquad B_m=Z^TG_m(P_m^TG_m)^{-1}, \tag{3.11}$$

where $\mathbf{a} \in \mathbb{R}^N$ is the reduced state and $\mathbf{f}_p(\mathbf{a})$ contains the selected nonlinear residual values. The offline stage precomputes B_m and the selected rows of Z. Online, the reduced state is reconstructed only at the selected locations, the nonlinear term is evaluated there, and the result is projected using B_m.

For the advection–diffusion model, define the projected mass and diffusion matrices by $M_N = Z^T M Z$ and $K_N = Z^T K Z$. The DEIM time step then has the form

$$M_N \frac{\mathbf{a}^{n+1} - \mathbf{a}^n}{\Delta t} + \frac{1}{\mathrm{Pe}} K_N \mathbf{a}^n + B_m \mathbf{f}_p(\mathbf{a}^n) = 0. \tag{3.12}$$

This preserves the reduced Galerkin structure while replacing the full-order nonlinear evaluation with a sampled one. For local operators, the online work can scale with the number of selected locations m and reduced dimension N, rather than the full mesh size $\mathcal{N}$. The precise cost depends on the nonlinear operator and whether its values can truly be evaluated from local data.

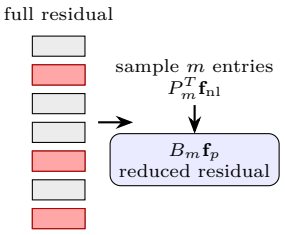


Figure 3.1: DEIM selects a small set of nonlinear residual entries, then maps them to the reduced equations.

The workflow has two phases:

- **Offline**: build the state basis Z and nonlinear basis G_m from snapshots; select the DEIM indices; precompute reduced operators, the interpolation map B_m, and the required rows of Z.
- **Online**: reconstruct only the sampled state values, evaluate the nonlinear residual at those locations, project it with B_m, and advance the reduced coefficients.

The added approximation introduces a trade-off: increasing m can improve the nonlinear representation but also raises online cost. Accuracy depends both on how well G_m captures the nonlinear snapshots and on the conditioning of $P_m^T G_m$; the speed-up is strictly dependent on the samples and basis choices.

3.4 Reduced Models for Navier–Stokes

The same ideas extend from the scalar advection model to incompressible flow. For velocity $\mathbf{u}$, pressure p, and kinematic viscosity ν, the parametrized Navier–Stokes equations are

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} - \nu \Delta \mathbf{u} + \nabla p = 0, \qquad \nabla \cdot \mathbf{u} = 0. \tag{3.13}$$

Following the FEM-to-ODE construction in Section 1.5, spatial discretization gives a saddle-point system:

$$M\dot{\mathbf{u}} + \nu K \mathbf{u} + C(\mathbf{u})\mathbf{u} + B^T \mathbf{p} = 0, \qquad B\mathbf{u} = 0. \tag{3.14}$$

Here B represents the discrete divergence constraint. The convection remains nonlinear, while velocity and pressure are coupled through the constraint.

An intrusive ROM builds POD bases for both fields, $\mathbf{u} \approx Z_u \mathbf{a}$ and $\mathbf{p} \approx Z_p \mathbf{q}$, and projects Equation (3.14):

$$Z_u^T M Z_u \dot{\mathbf{a}} + \nu Z_u^T K Z_u \mathbf{a} + Z_u^T C(Z_u \mathbf{a}) Z_u \mathbf{a} + Z_u^T B^T Z_p \mathbf{q} = 0, \tag{3.15}$$

$$Z_p^T B Z_u \mathbf{a} = 0. \tag{3.16}$$

This is the same projection principle used for the reduced system in Equation (2.7), now with a velocity–pressure constraint.

3.5 Non-Intrusive ROMs

Projection-based ROMs use the governing discrete equations, as described in Section 2.2. They are physics-based, use known equations, and we have control over them. However, in many cases you do not know the details of the FOM, and the choices you adopt could bring to stability issues and hidden problems.
A **non-intrusive ROM (NIROM)** instead learns from full-order snapshots and does not need online access to the FOM operators. Its basic idea is to compress each snapshot with POD, then learn how the reduced coefficients depend on the parameters.

For training parameters $\boldsymbol{\mu}_i$, let $\mathbf{u}_{\mathcal{N}}(\boldsymbol{\mu}_i)$ be the corresponding full-order solutions and let Z be an M-orthonormal POD basis, as in Section 2.3. Their reduced coordinates are

$$\mathbf{a}_i = Z^T M \mathbf{u}_{\mathcal{N}}(\boldsymbol{\mu}_i), \qquad i = 1, \dots, n_{\text{train}}. \tag{3.17}$$

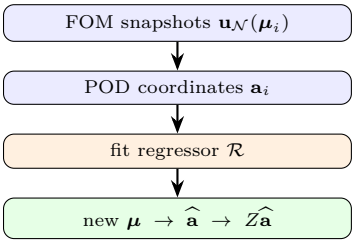


Figure 3.2: A NIROM learns a parameter-to-coordinate map offline and reconstructs a field online.

A regressor $\mathcal{R}$ maps a new parameter to estimated coordinates, and the corresponding field is reconstructed as

$$\widehat{\mathbf{a}}(\boldsymbol{\mu}) = \mathcal{R}(\boldsymbol{\mu}), \qquad \widehat{\mathbf{u}}_{\mathcal{N}}(\boldsymbol{\mu}) = Z\widehat{\mathbf{a}}(\boldsymbol{\mu}). \tag{3.18}$$

For time-dependent problems, time can also be an input, $(\boldsymbol{\mu}, t) \mapsto \widehat{\mathbf{a}}$. Alternatively, a learned evolution map can advance the reduced state, $\mathbf{a}^{n+1} = \mathcal{F}(\mathbf{a}^n, \boldsymbol{\mu})$; errors in this map may accumulate over repeated time steps.

3.5.1 Regression Models

The slides compare three ways to learn the reduced coordinates:

- **Radial basis function (RBF) interpolation**: represent each coordinate as a weighted sum of kernels centred at training parameters, such as Gaussian or thin-plate-spline kernels. For coordinate k, an RBF model has the form

$$\widehat{a}_k(\boldsymbol{\mu}) = \sum_{i=1}^{n_{\text{train}}} w_{ik}\, \phi(\|\boldsymbol{\mu} - \boldsymbol{\mu}_i\|), \qquad \Phi W = A, \quad \Phi_{ij} = \phi(\|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|). \tag{3.19}$$

  Here Φ is the kernel matrix, W contains the interpolation weights, and A contains the training POD coordinates. The interpolant matches the training data. Online evaluation is inexpensive, but the kernel matrix can be ill-conditioned and performance may degrade as parameter dimension grows. RBF models do not provide an uncertainty estimate by default.

- **Gaussian process regression (GPR)**: model each coordinate as a Gaussian process. For coordinate k, let $\mathbf{y}_k = (a_{1k}, \dots, a_{n_{\text{train}}k})^T$ be its training values, let $(K_{\text{GP}})_{ij} = k(\boldsymbol{\mu}_i, \boldsymbol{\mu}_j)$, and let $\mathbf{k}_*$ contain the kernel values between a query and the training parameters. The predictive mean and variance are

$$\begin{aligned} m_k(\boldsymbol{\mu}_*) &= \mathbf{k}_*^T (K_{\text{GP}} + \sigma_n^2 I)^{-1} \mathbf{y}_k, \\ s_k^2(\boldsymbol{\mu}_*) &= k(\boldsymbol{\mu}_*, \boldsymbol{\mu}_*) - \mathbf{k}_*^T (K_{\text{GP}} + \sigma_n^2 I)^{-1} \mathbf{k}_*. \end{aligned} \tag{3.20}$$

  GPR provides a predictive uncertainty and can work well with few snapshots, but exact training scales cubically with the number of training

points and requires quadratic storage. A stationary kernel may also smooth away sharp changes.

- **Artificial neural network (ANN)**: train a feed-forward network to map parameters directly to POD coordinates by minimizing the snapshot mean-squared error,

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} \|\mathbf{a}_i - \text{NN}_{\boldsymbol{\theta}}(\boldsymbol{\mu}_i)\|_2^2. \tag{3.21}$$

  An ANN can represent strongly nonlinear parameter maps and evaluate quickly with one forward pass, but usually needs more data and careful validation; training is non-convex and uncertainty is not built in.

Aspect	RBF	GPR	ANN
Data demand	Low	Low	Medium to high
Training	Linear solve	Cubic cost plus kernel tuning	Iterative, non-convex
Online evaluation	Kernel evaluations	Kernel evaluations and variance	One forward pass
Uncertainty	Not native	Predictive variance	Not native(ensembles are possible)
Main limitation	Conditioning; high-dimensional inputs	Training cost; kernel assumptions	Data demand and tuning

As a rule of thumb, RBF or GPR models suit a small training set and a smooth parameter map; ANNs are often more suitable when there are many parameters or snapshots and enough data to train them.

3.6 Error and Validation

NIROM accuracy has two main sources: the POD space may omit important solution features, and the regressor may predict the retained coordinates inaccurately. If the columns of Z are orthonormal in the M inner product and $\mathbf{a} = Z^T M \mathbf{u}_{\mathcal{N}}$, these contributions are orthogonal:

$$\|\mathbf{u}_{\mathcal{N}} - Z\widehat{\mathbf{a}}\|_M^2 = \|\mathbf{u}_{\mathcal{N}} - ZZ^T M \mathbf{u}_{\mathcal{N}}\|_M^2 + \|\mathbf{a} - \widehat{\mathbf{a}}\|_2^2. \tag{3.22}$$

The first term is the projection error, controlled by the retained modes and POD eigenvalue decay as in Equation (2.24). The second is the regression error, controlled by the model and the parameter samples. Increasing the number of POD modes can reduce projection error, but total error eventually levels off when regression error dominates.

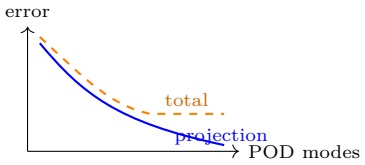


Figure 3.3: Total error can plateau when regression error dominates.

Sample parameter space deliberately, for example with Latin hypercube or Sobol points; greedy selection or GPR variance can guide adaptive refinement. Reserve separate training, validation, and test sets. Use validation data to tune model size and hyperparameters, and keep the test set untouched until the final assessment. Report a relative full-state error on unseen parameters alongside the POD projection error, so the two sources of error remain distinguishable.

3.7 Summary

Hyper-reduction retains the physical equations and addresses the nonlinear online bottleneck by approximating residuals at selected locations. DEIM can substantially reduce cost, but its accuracy depends on the nonlinear basis and sample selection. For incompressible flows, ROMs must additionally handle the velocity–pressure constraint and possible loss of stability from discarded scales. Non-intrusive methods avoid online access to the FOM by learning a parameter-to-latent-state map; their usefulness depends on snapshot coverage, regression quality, and honest validation. In both approaches, a small reduced state alone does not guarantee either fast evaluation or reliable predictions.